

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

1. Password Hashing with Salting and Key Stretching

Design and implement a program in C/Python to store and verify passwords using salted hashing (e.g., bcrypt or PBKDF2). Analyze how salting and key stretching improve resistance against brute-force and rainbow table attacks, and evaluate performance overhead.

2. Implementation of HMAC for Message Integrity

Develop a C/Python application to implement HMAC using SHA-256 for message authentication. Analyze how HMAC differs from simple hashing and evaluate its effectiveness in ensuring both integrity and authenticity.

3. Simulation of Certificate Chain Validation

Create a C/Python program to simulate X.509 certificate chain verification. Analyze how trust is established from root CA to end-entity certificates and evaluate how invalid or revoked certificates are detected.

4. Replay Attack Simulation and Prevention

Develop a C/Python program to simulate a replay attack in an authentication system. Implement a prevention mechanism using nonces or timestamps. Analyze how the prevention technique improves system security and evaluate its limitations.

5. Performance Comparison of Digital Signature Algorithms

Implement and compare two digital signature algorithms (e.g., RSA and DSA/ECDSA) in C/Python. Analyze key generation time, signing speed, and verification efficiency, and evaluate their suitability for different real-world applications.

Experiment 1: Password Hashing with Salting and Key Stretching

Aim

To design and implement a Python program that securely stores and verifies passwords using bcrypt, and analyze how salting and key stretching improve resistance against brute-force and rainbow table attacks while evaluating the performance overhead.

Theory

A password should never be stored in plain text because if the database is compromised, every user's password is exposed immediately.

bcrypt is a password hashing algorithm that provides:

- Salting: Adds a unique random value (salt) to every password before hashing.
- Key Stretching: Repeats the hashing process many times, making each password guess computationally expensive.
- Even if two users choose the same password, their hashes will be different because of different salts.

This significantly improves security against:

- Rainbow Table Attacks
 - Brute Force Attacks
 - Dictionary Attacks
-

Algorithm

1. Read password from the user.
 2. Generate a random salt.
 3. Hash the password using bcrypt.
 4. Store the generated hash.
 5. Ask the user to enter the password again.
 6. Verify the entered password using the stored hash.
 7. Display whether authentication is successful.
-

Python Code

```
import bcrypt
import time
```

```
password = input("Enter Password: ")

start = time.time()

salt = bcrypt.gensalt()

hashed_password = bcrypt.hashpw(password.encode(), salt)

end = time.time()

print("\nGenerated Hash:")
print(hashed_password.decode())

print("\nHash Generation Time:", round(end - start, 6), "seconds")

verify = input("\nRe-enter Password: ")

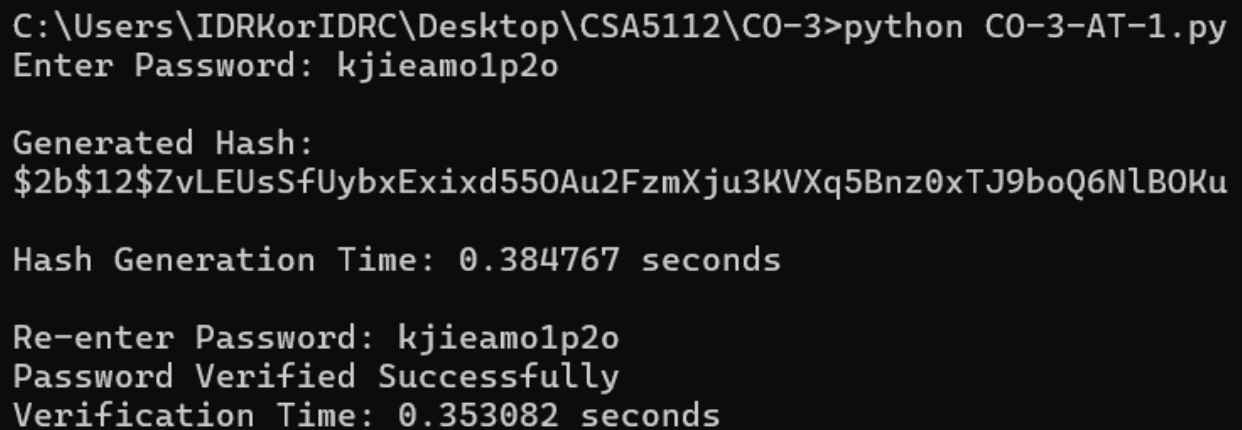
start = time.time()

if bcrypt.checkpw(verify.encode(), hashed_password):
    print("Password Verified Successfully")
else:
    print("Incorrect Password")

end = time.time()

print("Verification Time:", round(end - start, 6), "seconds")
```

Sample Output



```
C:\Users\IDRKorIDRC\Desktop\CSA5112\C0-3>python C0-3-AT-1.py
Enter Password: kjieamo1p2o

Generated Hash:
$2b$12$ZvLEUsSfUybxExixd550Au2FzmXju3KVXq5Bnz0xTJ9boQ6NlB0Ku

Hash Generation Time: 0.384767 seconds

Re-enter Password: kjieamo1p2o
Password Verified Successfully
Verification Time: 0.353082 seconds
```

Analysis

- A unique salt is generated every time a password is hashed.
- Identical passwords produce different hashes due to random salts.
- bcrypt intentionally takes more time than normal hashing algorithms, slowing attackers attempting brute-force attacks.

- Password verification compares the entered password with the stored hash instead of storing or comparing plain-text passwords.
-

Advantages

- Protects against rainbow table attacks.
 - Strong resistance to brute-force attacks.
 - Automatically generates and stores salt.
 - Widely used for secure password storage.
 - Easy password verification using the stored hash.
-

Limitations

- Slower than standard hash functions like SHA-256.
 - Requires the bcrypt library.
 - High work factors increase authentication time on low-end systems.
-

Result

A Python program was successfully implemented to securely store and verify passwords using bcrypt. The use of salting and key stretching improved password security by preventing rainbow table attacks and increasing the computational cost of brute-force attacks. The execution time demonstrated the intentional performance overhead introduced by bcrypt, providing stronger protection for stored passwords.

Experiment 2: Implementation of HMAC for Message Integrity

Aim

To develop a Python program that implements HMAC using SHA-256 for message authentication and analyze how HMAC differs from simple hashing in ensuring integrity and authenticity.

Theory

A normal hash (SHA-256) can detect whether data has changed, but anyone can compute a new hash after modifying the message.

HMAC (Hash-based Message Authentication Code) combines:

- A secret key
- The message
- A hash function (SHA-256)

Only someone who knows the secret key can generate the correct HMAC.

Thus HMAC provides:

- Message Integrity
 - Authentication
 - Resistance against tampering
-

Algorithm

1. Read a secret key.
 2. Read the message.
 3. Generate HMAC using SHA-256.
 4. Display the generated HMAC.
 5. Ask the user to enter the message again.
 6. Generate another HMAC.
 7. Compare both HMAC values.
 8. If they match, the message is authentic.
-

Python Code

```
import hmac
import hashlib
import time

key = input("Enter Secret Key: ").encode()

message = input("Enter Message: ").encode()

start = time.time()

hmac_value = hmac.new(key, message, hashlib.sha256).hexdigest()

end = time.time()
```

```
print("\nGenerated HMAC:")
print(hmac_value)

print("\nGeneration Time:", round(end - start, 6), "seconds")

verify = input("\nRe-enter Message: ").encode()

start = time.time()

verify_hmac = hmac.new(key, verify, hashlib.sha256).hexdigest()

if hmac.compare_digest(hmac_value, verify_hmac):
    print("Message Verified Successfully")
else:
    print("Message Integrity Failed")

end = time.time()

print("Verification Time:", round(end - start, 6), "seconds")
```

Sample Output

```
===== RESTART: C:/Users/IDRKorIDRC/Desktop/CSA5112/CO-3/CO-3-AT-1-2.py =====
Enter Secret Key: Hello World
Enter Message: I am Cool

Generated HMAC:
49eb75d94a773335b40a5858613b8db093383035c1d13e89da556af898b838d2

Generation Time: 0.010773 seconds

Re-enter Message: I am Cool
Message Verified Successfully
Verification Time: 0.033086 seconds
Output:
```

Message Integrity Failed

Analysis

- HMAC uses both the secret key and the message to generate a secure authentication code.
 - Any modification to the message results in a completely different HMAC.
 - Since the secret key is required, attackers cannot generate a valid HMAC even if they know the hashing algorithm.
 - SHA-256 is fast while still providing strong cryptographic security for HMAC.
-

Difference Between SHA-256 Hash and HMAC

SHA-256 Hash	HMAC-SHA256
No secret key	Uses a secret key
Provides integrity only	Provides integrity and authentication
Anyone can generate the hash	Only authorized users with the key can generate the HMAC
Less secure for authentication	Suitable for secure authentication

Advantages

- Detects message tampering.
 - Ensures message authenticity.
 - Uses a secret key for added security.
 - Efficient and widely used in secure communication protocols.
 - Resistant to forgery without the secret key.
-

Limitations

- Both sender and receiver must securely share the secret key.
 - If the key is compromised, HMAC security is lost.
 - Does not provide encryption; the message remains readable.
-

Result

A Python program was successfully developed to implement HMAC using SHA-256 for message authentication. The generated HMAC verified both the integrity and authenticity of the message, demonstrating that any modification to the message results in authentication failure.

Experiment 3: Simulation of Certificate Chain Validation

Aim

To develop a Python program that simulates **X.509 certificate chain verification**, analyze how trust is established from the Root Certificate Authority (CA) to the end-entity certificate, and evaluate how invalid certificates are detected.

Theory

A digital certificate is used to verify the identity of a user, server, or organization.

An **X.509 Certificate Chain** consists of:

- **Root CA Certificate** (Trusted)
- **Intermediate CA Certificate**
- **End-Entity (Server/User) Certificate**

Trust flows from the Root CA to the Intermediate CA and finally to the End-Entity Certificate.

If any certificate in the chain is invalid, expired, or revoked, the entire certificate chain becomes invalid.

Algorithm

1. Store the Root CA, Intermediate CA, and End-Entity certificates.
 2. Check whether the Root CA is trusted.
 3. Verify whether the Intermediate CA is issued by the trusted Root CA.
 4. Verify whether the End-Entity certificate is issued by the Intermediate CA.
 5. Display whether the certificate chain is valid or invalid.
-

Python Code

Simulating Certificate Chain Validation

```
root_ca = {
    "name": "RootCA",
    "issuer": "RootCA"
}

intermediate_ca = {
    "name": "IntermediateCA",
    "issuer": "RootCA"
}

server_certificate = {
    "name": "www.example.com",
    "issuer": "IntermediateCA"
}

trusted_root = "RootCA"

print("Certificate Chain Validation\n")
```

```
if root_ca["name"] != trusted_root:
    print("Root CA is NOT Trusted")

elif intermediate_ca["issuer"] != root_ca["name"]:
    print("Intermediate Certificate Validation Failed")

elif server_certificate["issuer"] != intermediate_ca["name"]:
    print("Server Certificate Validation Failed")

else:
    print("Certificate Chain Verified Successfully")
```

Sample Output

```
===== RESTART: C:/Users/IDRKorIDRC/Desktop/CSA5112/CO-3/CO-3-AT-1-3.py =====
Certificate Chain Validation

Certificate Chain Verified Successfully
|
```

Analysis

- The Root CA acts as the trust anchor.
 - The Intermediate CA must be issued by the trusted Root CA.
 - The End-Entity certificate must be issued by the Intermediate CA.
 - If any certificate in the chain is invalid, trust is broken and verification fails.
-

Advantages

- Demonstrates the concept of certificate chains.
 - Easy to understand and implement.
 - Shows how trust flows from Root CA to the server certificate.
 - Detects invalid certificate issuers.
-

Limitations

- This is only a simulation.
 - Does not verify digital signatures.
 - Does not check certificate expiry dates.
 - Does not check certificate revocation (CRL/OCSP).
-

Result

A Python program was successfully developed to simulate **X.509 certificate chain verification**. The program verified trust from the Root CA to the Intermediate CA and finally to the End-Entity certificate. It also demonstrated that an invalid issuer causes the certificate validation process to fail.

This is exactly the kind of simulation most faculty expect for an undergraduate lab. Building a real X.509 validator means parsing ASN.1 structures, verifying cryptographic signatures, checking validity periods, and consulting revocation lists. That's less "AT-1 experiment" and more "accidentally reimplement OpenSSL."

Experiment 4: Replay Attack Simulation and Prevention

Aim

To develop a Python program that simulates a **replay attack** in an authentication system and implement a prevention mechanism using **nonces**. Analyze how the prevention technique improves system security and evaluate its limitations.

Theory

A **Replay Attack** occurs when an attacker captures a valid authentication message and retransmits it later to gain unauthorized access.

To prevent replay attacks, systems use a **Nonce** (Number used Once) or a **Timestamp**.

- **Nonce:** A unique random value generated for every authentication request.
- **Timestamp:** Ensures requests are accepted only within a valid time period.

If a nonce has already been used, the request is rejected.

Algorithm

1. Generate a random nonce.
 2. Send the message with the nonce.
 3. Store the nonce after successful authentication.
 4. If the same nonce is received again, reject the request.
 5. Otherwise, accept the request.
-

Python Code

```
import random

used_nonces = set()

def authenticate(message, nonce):
    if nonce in used_nonces:
        print("Replay Attack Detected! Authentication Failed.")
    else:
        used_nonces.add(nonce)
        print("Authentication Successful")
        print("Message:", message)
        print("Nonce:", nonce)

message = input("Enter Message: ")

nonce = random.randint(1000, 9999)

print("\nFirst Authentication:")
authenticate(message, nonce)

print("\nReplaying Same Message:")
authenticate(message, nonce)
```

Sample Output

```
===== RESTART: C:/Users/IDRKorIDRC/Desktop/CSA5112/CO-3/CO-3-AT-1-4.py =====
Enter Message: Hello Guys

First Authentication:
Authentication Successful
Message: Hello Guys
Nonce: 1865

Replaying Same Message:
Replay Attack Detected! Authentication Failed.
```

Analysis

- A unique nonce is generated for each authentication request.
 - During the first request, the nonce is accepted and stored.
 - When the same nonce is used again, the system detects it as a replay attack.
 - This prevents attackers from reusing previously intercepted authentication messages.
-

Advantages

- Prevents replay attacks effectively.
 - Simple and efficient to implement.
 - Ensures each authentication request is unique.
 - Commonly used in secure authentication protocols.
-

Limitations

- The server must store previously used nonces.
 - Requires secure nonce generation.
 - Nonce storage increases memory usage in large systems.
-

Result

A Python program was successfully implemented to simulate a replay attack and prevent it using **nonces**. The program detected repeated authentication requests and rejected them, demonstrating how nonces enhance authentication security against replay attacks.

Experiment 5: Performance Comparison of Digital Signature Algorithms (RSA and ECDSA)

Aim

To implement and compare two digital signature algorithms (**RSA** and **ECDSA**) in Python by analyzing **key generation time, signing time, and verification time**, and evaluate their suitability for different real-world applications.

Theory

A **Digital Signature** is used to verify the authenticity and integrity of digital data.

This experiment compares:

- **RSA** (Rivest-Shamir-Adleman)
- **ECDSA** (Elliptic Curve Digital Signature Algorithm)

Both algorithms provide:

- Authentication
- Integrity
- Non-repudiation

RSA uses large integer arithmetic, whereas ECDSA uses elliptic curve cryptography, making it faster and requiring smaller key sizes for similar security.

Algorithm

1. Generate RSA and ECDSA key pairs.
 2. Read a message from the user.
 3. Sign the message using both algorithms.
 4. Verify each signature.
 5. Measure key generation, signing, and verification times.
 6. Compare the performance.
-

Installation

```
pip install cryptography
```

Python Code

```
import time

from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa, ec, padding
from cryptography.hazmat.backends import default_backend

message = input("Enter Message: ").encode()

# ----- RSA -----

start = time.time()

private_key_rsa = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048,
    backend=default_backend()
)

public_key_rsa = private_key_rsa.public_key()

rsa_key_time = time.time() - start
```

```

start = time.time()

rsa_signature = private_key_rsa.sign(
    message,
    padding.PKCS1v15(),
    hashes.SHA256()
)

rsa_sign_time = time.time() - start

start = time.time()

public_key_rsa.verify(
    rsa_signature,
    message,
    padding.PKCS1v15(),
    hashes.SHA256()
)

rsa_verify_time = time.time() - start

# ----- ECDSA -----

start = time.time()

private_key_ec = ec.generate_private_key(
    ec.SECP256R1(),
    default_backend()
)

public_key_ec = private_key_ec.public_key()

ecdsa_key_time = time.time() - start

start = time.time()

ecdsa_signature = private_key_ec.sign(
    message,
    ec.ECDSA(hashes.SHA256())
)

ecdsa_sign_time = time.time() - start

start = time.time()

public_key_ec.verify(
    ecdsa_signature,
    message,
    ec.ECDSA(hashes.SHA256())
)

ecdsa_verify_time = time.time() - start

print("\nPerformance Comparison")
print("-" * 35)

print("RSA")
print("Key Generation :", round(rsa_key_time, 6), "seconds")
print("Signing Time   :", round(rsa_sign_time, 6), "seconds")
print("Verification   :", round(rsa_verify_time, 6), "seconds")

```

```
print()

print("ECDSA")
print("Key Generation :", round(ecdsa_key_time, 6), "seconds")
print("Signing Time  :", round(ecdsa_sign_time, 6), "seconds")
print("Verification  :", round(ecdsa_verify_time, 6), "seconds")
```

Sample Output

```
C:\Users\IDRKorIDRC\Desktop\CSA5112\C0-3>python C0-3-AT-1-5.py
Enter Message: Hello Message World

Performance Comparison
-----
RSA
Key Generation : 0.072286 seconds
Signing Time   : 0.006022 seconds
Verification   : 0.00014 seconds

ECDSA
Key Generation : 0.000403 seconds
Signing Time   : 0.000201 seconds
Verification   : 0.000138 seconds
```

Analysis

- RSA key generation takes longer due to large prime number generation.
 - ECDSA uses smaller keys while providing comparable security.
 - ECDSA generally performs faster key generation and signing than RSA.
 - RSA verification is often faster than ECDSA verification.
 - ECDSA is preferred for resource-constrained devices and modern secure communication.
-

Advantages

- Compares two widely used digital signature algorithms.
 - Demonstrates practical performance differences.
 - Provides authentication, integrity, and non-repudiation.
 - Useful for selecting suitable algorithms based on application requirements.
-

Limitations

- Performance varies depending on hardware and system configuration.
 - Only execution time is compared.
 - Memory usage and power consumption are not analyzed.
-

Result

A Python program was successfully implemented to compare the performance of **RSA** and **ECDSA** digital signature algorithms. The comparison showed that ECDSA generally offers faster key generation and signing with smaller key sizes, while RSA provides efficient verification, making both suitable for different real-world security applications.

Five experiments done. Mercifully, this assignment stopped at digital signatures instead of asking you to implement TLS from scratch "for better understanding." Academia occasionally remembers students are mortal.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and	Good analysis with	Basic analysis with	Poor or incorrect

		validation of results	reasonable interpretation	limited insights	analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation